RAG PROJECT NOTES

A beginner-friendly guide to Retrieval-Augmented Generation
Python • LangChain • Ollama • Chroma • Streamlit

Learning goal
Understand how a RAG application loads documents, searches for relevant information, and uses a language model to answer questions grounded in that information.

What you will learn

• RAG concepts and end-to-end workflow

• Document loaders, chunking, embeddings, vector stores, and retrievers

• A small local RAG example using Ollama and LangChain

• Troubleshooting common setup errors

• Evaluation, improvements, project ideas, and interview notes

Prepared as study notes for beginners and interview revision.

1. What is RAG?

RAG stands for Retrieval-Augmented Generation. It is a technique that combines information retrieval with a large language model (LLM). When a user asks a question, the application searches an external knowledge source for relevant information, then gives that information to the LLM to help generate an answer. This is useful for private documents, internal knowledge bases, manuals, policies, and frequently updated information.

Why use RAG?

• Answer questions using your own documents or knowledge base.

• Provide relevant context that may not be present in the model's training data.

• Make answers easier to verify by showing source documents or page references.

• Reduce some hallucinations when retrieval is relevant and the model follows the supplied context.

Important: RAG does not guarantee correctness. Poor retrieval, outdated documents, ambiguous questions, or an LLM ignoring context can still produce wrong answers.

2. The RAG workflow

Step	What happens
1. Load	Read PDF, TXT, DOCX, HTML, or other supported source files.
2. Split	Divide long text into smaller chunks, often with a small overlap.
3. Embed	Convert each chunk into a numerical vector using an embedding model.
4. Store	Save vectors and associated text/metadata in a vector store.
5. Retrieve	Embed or process the user query and search for relevant chunks.
6. Generate	Pass the question and retrieved context to the LLM for an answer.

3. Key components

Document loader: Reads source content and often stores metadata such as filename or page number.

Text splitter: Creates manageable chunks. Chunk size and overlap should be tested against your documents.

Embedding model: Maps text to vectors so semantically related text can be compared. Example: nomic-embed-text.

Vector store: Stores vectors and supports similarity search. Examples: Chroma and FAISS; managed options include Pinecone.

Retriever: Returns candidate chunks for a question. A basic retriever often uses vector similarity.

LLM: Uses the question and retrieved context to generate a natural-language response. Example: Llama 3.2 via Ollama.

Prompt template: Defines how context and question are presented and gives instructions for grounded answers.

4. Essential terminology

Chunk size: The target amount of text in each chunk.

Chunk overlap: Text repeated between adjacent chunks to reduce loss of context at boundaries.

Top-k: Number of candidate chunks returned by a retriever, e.g. k=3.

Similarity search: Finds vectors close to the query vector according to a similarity/distance measure.

Metadata filtering: Restricts retrieval using attributes such as document type, department, date, or access level.

Reranking: Reorders retrieved candidates with a model or scoring method to put the most relevant results first.

Hybrid search: Combines semantic/vector retrieval with keyword-based retrieval.

5. A small local RAG project

This example reads a local text file, splits it into chunks, embeds the chunks with Ollama, stores them in Chroma, retrieves relevant context, and asks a local chat model to answer. After installing the models and packages, inference does not require a paid hosted API. The first model download requires an internet connection.

Install packages

```
python -m pip install -U langchain langchain-ollama langchain-chroma langchain-text-splitters
```

Download models in PowerShell

```
ollama pull llama3.2
ollama pull nomic-embed-text
```

Create a file named knowledge.txt

```
RAG stands for Retrieval-Augmented Generation.
RAG retrieves relevant information and provides it to a language model.
Ollama can run supported language models locally.
```

Save as main.py

```
from langchain_ollama import ChatOllama, OllamaEmbeddings
from langchain_chroma import Chroma
from langchain_text_splitters import RecursiveCharacterTextSplitter
from pathlib import Path

# Read the file relative to this script
file_path = Path(__file__).resolve().parent / "knowledge.txt"
text = file_path.read_text(encoding="utf-8")

# Split text into chunks
splitter = RecursiveCharacterTextSplitter(
chunk_size=300,
chunk_overlap=50
)
chunks = splitter.create_documents([text])

# Embed and store chunks locally
embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(
documents=chunks,
embedding=embeddings
)

# Retrieve relevant context
retriever = vectorstore.as_retriever(
search_kwargs={"k": 3}
)
question = "What does RAG do?"
docs = retriever.invoke(question)
context = "\n\n".join(d.page_content for d in docs)

# Generate a context-grounded answer
llm = ChatOllama(model="llama3.2", temperature=0)
prompt = f"""Answer using only the context below.
If the answer is not present, say you do not know.

Context:
{context}

Question: {question}
Answer:"""

response = llm.invoke(prompt)
print(response.content)
```

Run: Open a terminal in the project folder and execute `python main.py`. Keep the Ollama app/service running. This starter demo creates a temporary vector store each run; use a persistent Chroma directory for a reusable project.

6. Troubleshooting

Problem	What to check
'ollama' is not recognized	Install Ollama for Windows, reopen the terminal, and verify PATH. Test with: ollama --version.
Model not found	Pull the exact model name used in code: ollama pull llama3.2.
Connection refused	Start the Ollama application/service and retry.
FileNotFoundError	Put knowledge.txt next to main.py or use an explicit path. Path(__file__).resolve().parent helps resolve relative to the script.
No module named ...	Activate the intended virtual environment and install the missing package using python -m pip.
Weak or irrelevant answers	Inspect retrieved chunks, adjust chunking/top-k, improve source quality, or test hybrid search/reranking.

7. Improving a RAG system

• **Improve source quality:** remove duplicate, irrelevant, or outdated documents and preserve useful metadata.

• **Test chunking:** compare chunk sizes and overlap using representative questions instead of assuming one setting is best.

• **Improve retrieval:** use metadata filters, hybrid search, query rewriting, or reranking when justified.

• **Ground the prompt:** tell the model to use retrieved context and say when the answer is not available.

• **Show sources:** return document names, page numbers, or other metadata alongside answers.

• **Evaluate:** create a test set of questions with expected sources/answers and track changes after each modification.

• **Secure data:** enforce access controls before retrieval; do not retrieve documents a user is not allowed to see.

8. Evaluation concepts

Context precision: How much of the retrieved context is relevant.

Context recall: How much of the needed information was retrieved.

Faithfulness: Whether the answer is supported by the retrieved context.

Answer relevance: Whether the answer addresses the user's question.

Latency and cost: How long responses take and what infrastructure/API cost is incurred.

9. Project ideas

Beginner — PDF chatbot: Ask questions about one PDF and display the source page.

Intermediate — Company knowledge assistant: Search policies and FAQs with metadata filters and citations.

Advanced — Research assistant: Search multiple sources, rerank results, compare evidence, and cite references.

Portfolio additions: Add Streamlit UI, document upload, persistent storage, chat history, logging, tests, and evaluation.

10. Interview revision

Q: What is RAG?
A: A system that retrieves relevant external information and gives it to an LLM to help generate a grounded answer.

Q: Why chunk documents?
A: To retrieve smaller, relevant pieces instead of passing an entire long document.

Q: What are embeddings?
A: Numerical representations of text used for semantic comparison.

Q: What does a retriever do?
A: Returns candidate document chunks relevant to the query.

Q: Does RAG eliminate hallucinations?
A: No. It can reduce them, but retrieval and generation still need evaluation.

Q: RAG vs fine-tuning?
A: RAG supplies external context at query time; fine-tuning updates model behavior/weights through training.

One-line summary

RAG retrieves relevant external knowledge and provides it to an LLM so the model can generate an answer grounded in that context.

Suggested learning order: text-file RAG → PDF loader → persistent vector store → Streamlit UI → source citations → retrieval evaluation.